

Aula Revisão de Programação Básica

Curso: Sistemas de Informação

Disciplina: Algoritmos e Estruturas de Dados

Professor: Fábio Leandro Rodrigues Cordeiro

- Algoritmos
- Estrutura de dados
- Programas
- Conceitos importantes
- Tipos de estruturas
- Vetores (arranjos) e Arranjos Multidimensionais
- Funções e Procedimentos
- Parâmetros
- Classes e Objetos
- Objetos em C#
- Modificadores de Acesso

Um algoritmo corresponde a uma descrição de um padrão de comportamento, expresso em termos de um conjunto finito de ações ([Dijkstra](#), 1971).

- Um algoritmo é uma sequência de passos finitos para resolução de um problema.
- Algoritmos fazem parte do dia a dia:
 - Instruções para uso de medicamentos
 - Manual para montar um aparelho
 - Receita de culinária





- Um algoritmo é dito correto se, para cada entrada, ele gera uma saída esperada.
- Um algoritmo pode ser especificado em uma linguagem comum, como um programa de computador ou projeto de hardware.

- Conjunto de dados que representam uma situação real
 - Abstração da Realidade
- Estruturas de dados e algoritmos estão intimamente ligados:
 - Não se pode estudar estruturas de dados sem considerar os algoritmos associados a elas.
 - Assim como a escolha dos algoritmos em geral depende da representação e da estrutura dos dados.
- É um meio para armazenar e organizar dados com o objetivo de facilitar o acesso e as modificações.
 - Exemplo: vetor ou matriz



- Programar é basicamente estruturar dados e construir algoritmos.
- Programas são formulações concretas de algoritmos abstratos, baseados em representações e estruturas específicas de dados.
- Programas representam uma classe especial de algoritmos capazes de serem seguidos por computadores.



- Um computador só é capaz de seguir programas em linguagem de máquina.
- É necessário construir linguagens mais adequadas, que facilitem a tarefa de programar um computador.
- Uma linguagem de programação é uma técnica de notação para programar.

Constantes:

- Representam valores inalteráveis durante toda a execução.

Variável:

- Espaço da memória reservado para armazenar um certo tipo de dado e tendo um nome para referenciar o seu conteúdo.
- Este valor pode variar durante a execução

Tipo de dados:

- Define um conjunto de valores que uma variável pode armazenar
- Define o conjunto de operações que pode ser executado com essa variável (ex.: int, double, string).
- Forma como a variável é armazenada e interpretada.

Tipo de variáveis:

Informa a quantidade de memória, em bytes, que a variável ocupará, e a forma como o valor deverá ser armazenado e interpretado.

int: números inteiros de 32 bits (4 bytes).

long: números inteiros (intervalo maior) de 64 bits (8 bytes).

double: ponto flutuante (maior precisão) 64 bits (8 bytes).

string: sequência de caracteres de 16 bits
(2 bytes) por caractere.

bool: booleano de 8 bits (1 bytes):



Conceitos Importantes

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Revisão
{
    class Program
    {
        static void Main(string[] args)
        {
            int idade = 25;
            double salario = 250.00, Salario = 100;
            int maximo = 65000;
            int GUESS = maximo;
            int orcamento_de_contabilidade_de_1998 = 1000;
        }
    }
}
```

Utilize nomes significativos para as variáveis

Exemplo:

`int idade;`

`float salario;`

Características dos nomes das variáveis

- Sempre começa com uma letra, ou um *underline*
- Com exceção do primeiro caractere, podem existir números
- Palavras-chaves não podem representar nomes de variáveis
- Caractere minúsculo $\neq$ caractere maiúsculo



Estrutura Condicional

Comando if

Forma Geral:

If (condição_for_verdadeira)
comando;

O comando if executa um teste utilizando um operador relacional do C#.

Condição_for_verdadeira

Pode ter apenas um operador relacional ou pode ter n operadores relacionais ligadas por operadores lógicos



Tipos de Estruturas

Estruturas de repetição:

Repetir comandos um número específico de vezes.

```
for (inicialização; teste; incremento)  
    comando(s);
```

Repetir comandos até que uma condição conhecida ocorra.

```
while (condição_for_verdadeira)  
    comando(s);
```

Executar comandos pelo menos uma vez, possivelmente repetindo-os no futuro

```
do{  
    comando(s);  
} while (condição_for_verdadeira);
```

Vetores (Arranjos)

- Um array (vetor) é uma sequencia não ordenada de elementos do mesmo tipo.
- Arranjos podem ser:
 - Unidimensional (Vetor)
 - Bidimensional (Matriz)
- Informalmente:
 - “arranjo é uma série de variáveis **do mesmo tipo** referenciadas por um único nome
 - cada variável (elemento do arranjo) é diferenciada por um índice

Vetores (Arranjos)

- Criando uma instância de array:
- Ao declarar uma variável de array seu tamanho não é declarado, isso ocorre no momento de instanciar um array utilizando *new*.
- Exemplo:
 - `int [] nota = new int [4];`
- Vetor de inteiros
- `nota [ 0 ]`, `nota [ 1 ]`, `nota [ 2 ]`, `nota[ 3 ]`

Vetores (Arranjos)

Quando o vetor é declarado, o compilador alocará memória suficiente para conter todos os elementos do vetor.

```
int [ ] peso = new int [10];
```

```
double [ ] nota = new double [41];
```

```
string [ ] nome = new string [80];
```

- É possível definir um vetor em que cada posição temos um outro vetor (matriz).

```
int [ , ] materia = new int[4,4]
```

- Interpretação:
temos 4 matérias, cada uma com 40 alunos

```
int [ , ] materia = new int[linha, coluna]
```



Funções e Procedimentos

- Conjunto de instruções para cumprir uma tarefa particular e agrupadas numa unidade com um nome para referenciá-la.
- Dividir a tarefa original em pequenas tarefas que simplificam e organizam o programa como um todo.
- Reduzir o tamanho e a complexidade do programa.
- Um programa pode conter uma ou mais funções.



- Chamar uma função é o meio pela qual solicitamos que o programa desvie o controle e passe à função, execute suas instruções e depois volte para a instrução seguinte que a chamou.
- Uma função pode conter zero ou mais argumentos.
- Funções retornam algum valor e procedimentos não.
- Duas funções podem ter o mesmo nome mas se diferem em relação aos parâmetros



- Programas podem passar dados para funções, chamados de parâmetros.
- Quando declarar uma função, deve ser declarado também os tipos dos parâmetros da funções, estes parâmetros estão entre parênteses.
- Uma função pode receber mais de um parâmetro, onde estes devem ser separados por vírgula.
- Funções podem não receber parâmetros algum



Classes

- As classes nos permitem organizar de forma coerente o código que criamos.
- Podemos criar uma classe chamada custos contendo todas as funções necessárias para manipular os cálculos referentes aos custos de uma empresa.
- Classes definem os dados e os comportamentos de um tipo de dados.



Classes

- Os membros de uma classe podem ser:

Construtores, Destrutores, Constantes, Campos, métodos, propriedades, indexadores, operadores, eventos, delegates, classes, interfaces, estruturas.



Classes

Uma classe é declarada com a instrução classe pode ser precedida de um modificador de acesso.

Sua sintaxe é:

```
[modificador] classe nome { }
```

Ex:

```
Public class Taxas {  
    //codigo aqui;  
}
```


- Para acessar os membros de uma classe, criamos uma instância de classe, chamada de objeto, que, por sua vez, é uma entidade concreta baseada em uma classe.
- Os objetos podem conter dados e ter comportamento: os dados dos objetos são armazenados em campos, propriedades e eventos do objeto.
- O comportamento dos objetos é definido por métodos e interface do objeto.
- Dois objetos idênticos podem não conter os mesmos dados



- LOUREIRO, Henrique. **C# 5.0 com Visual Studio® 2012**: curso completo. Lisboa: FCA, c2013. xxii, 585 p. ISBN 9789727227525 (Disponível no Acervo).
- MANZANO, José Augusto N. G; OLIVEIRA, Jayr Figueiredo de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 26. ed. rev. São Paulo: Érica, 2012. 328 p. ISBN 9788536502212 (Disponível no Acervo).
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. **Estruturas de dados e seus algoritmos**. 3. ed. Rio de Janeiro: LTC - Livros Técnicos e Científicos, c2010. xvi, 302 p. ISBN 9788521617501 (Disponível no Acervo).